Algorithmie

SAÉ 3.02 - Labyrinthe

Développement efficace

Étudiants :

Victor Bredelle
Antonin Marouze
Romain Harlaut
Angèl Zheng
Baptiste Lavogiez

Enseignant :

Mme Everaere

27 octobre 2025

Table des matières

1 Introduction **2**

2 Investissement dans le projet **3**

3 Structures de données pour les labyrinthes **5**
3.1 Labyrinthes aléatoires 5
3.2 Labyrinthes parfaits 5

4 Algorithmes de génération **6**
4.1 Labyrinthes aléatoires 6
4.1.1 Algorithme utilisé 6
4.1.2 Pseudo-code 6
4.1.3 Structures de données utilisées 6
4.1.4 Pertinence 6
4.2 Labyrinthes parfaits 6
4.2.1 Algorithme utilisé 6
4.2.2 Pseudo-code 6
4.2.3 Structures de données utilisées 6
4.2.4 Pertinence 6

5 Efficacité **7**
5.1 Méthode 7
5.1.1 Test de performance 7
5.1.2 Modélisation 7
5.2 Labyrinthes aléatoires 8
5.2.1 Test de performance 8
5.2.2 Visualisation 8
5.3 Labyrinthes parfaits 8
5.3.1 Test de performance 8
5.3.2 Visualisation 8
5.4 Conclusion 8

6 Complément et Bilan **9**

7 Conclusion **10**

1 Introduction

Ce rapport présente l'analyse UML du projet **SAÉ S3.02 - Labyrinthe**, un jeu avec deux modes (libre et progression) :

1. Diagrammes de cas d'utilisation
2. Fiches descriptives
3. Prototype d'interface
4. Diagramme de classe

2 Investissement dans le projet

Groupe G3.3

- Victor Bredelle
- Antonin Marouze
- Romain Harlaut
- Angèl Zheng
- Baptiste Lavogiez

Nom	Victor	Antonin	Romain	Angèl	Baptiste	Total
Part	⟨Data 2.1⟩	⟨Data 2.2⟩	⟨Data 2.3⟩	⟨Data 2.4⟩	⟨Data 2.5⟩	100%

Table 2.1 – Répartition de l'investissement

1. Victor

Rédaction de trois fiches descriptives avec Baptiste : Charger profil, Afficher progression, Lancer mode progression.

2. Antonin

Réalisation de la maquette *Figma* avec Angèl et Romain.

3. Romain

Réalisation de la fiche descriptive "Déplacer joueur" et aide à la maquette *Figma*.

4. Angèl

Réalisation de la maquette *Figma* avec Antonin et Romain.

5. Baptiste

Rédaction de trois fiches descriptives avec Victor et rédaction du rapport.

Le groupe

Information

L'ensemble des contributeurs a collaboré à l'élaboration du diagramme de classes.

3 Structures de données pour les labyrinthes

3.1 Labyrinthes aléatoires

3.2 Labyrinthes parfaits

4 Algorithmes de génération

4.1 Labyrinthes aléatoires

4.1.1 Algorithme utilisé

4.1.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
    faire tata
```

4.1.3 Structures de données utilisées

4.1.4 Pertinence

4.2 Labyrinthes parfaits

4.2.1 Algorithme utilisé

4.2.2 Pseudo-code

4.2.3 Structures de données utilisées

4.2.4 Pertinence

5 Efficacité

5.1 Méthode

5.1.1 Test de performance

> **1. Classe "Benchmark"**

java

```java
// Fichier: HelloWorld.java
public class HelloWorld {
/**
* Le point d'entrée du programme.
*/
    public static void main(String[] args) {
    System.out.println("Hello, World from Java!");
    }
}
```

5.1.2 Modélisation

> **2. Script "Modélisation"**

python

```python
# Simple script pour saluer le monde
def say_hello(name):
"""
Cette fonction affiche un message de salutation.
```

```
"""
print(f"Hello, {name}!")

if __name__ == "__main__":
say_hello("World")
```

5.2 Labyrinthes aléatoires

5.2.1 Test de performance

5.2.2 Visualisation

5.3 Labyrinthes parfaits

5.3.1 Test de performance

5.3.2 Visualisation

5.4 Conclusion

6 Complément et Bilan

7 Conclusion